DEF CON 34 DEMO LABS

Senrigan × Suzaku

千里眼 × 朱雀

Threat Hunting & DFIR for AWS

No SIEM. Just your laptop.



Yamato Security

Fukusuke Takahashi / Zach Mathis / Akira Nishikawa

Speaking for a much larger team



Fukusuke Takahashi

NTTDATA-CERT / NTTDATA

Development lead, DFIR tooling.



Zach Mathis

FOUNDER, YAMATO SECURITY

Founder and project leader.



Akira Nishikawa

AWS COMMUNITY BUILDER

Development lead, DFIR tooling.

Three of us on stage. Detections from Sigma, the AWS community, and many other public projects.

Yamato Security — based in Japan since 2012

- Started in 2012 by Zach Mathis to build a local security community in western Japan.
- Now providing training, CTF events and webinars for thousands of people across the country.
- Volunteer-run
- Provides free open-source DFIR tools, built in Japan and used worldwide.

2012
SINCE

1,000s
TRAINED IN JAPAN

3.3K+
★ ON HAYABUSA



Hayabusa
Windows event logs

Mecha Hayabusa
AI analyzer

Suzaku
CloudTrail timeline

Senrigan
CloudTrail hunting

← TODAY

Agenda

01	**The problem**	Why CloudTrail hunting is difficult
02	**The approach**	Offline, rule-driven, community-powered
03	**Suzaku**	Sigma detections → DFIR timeline
04	**Senrigan**	100+ hunts, 100+ charts, AI chat
05	**Live demo**	Raw logs → finished investigation
06	**Q & A**	Ask us anything

SECTION 01

The Problem

Your CloudTrail logs are already collected.
However, they are difficult to use as-is.



Your software supply chain is an attack path to AWS

MAR 2026

UNC6426

Stolen GitHub token, abused OIDC trust — AWS admin in 72 hours.

MAY 2026

Shai-Hulud returns

A worm spreading across 170+ npm and PyPI packages, hunting cloud keys.

MAY 2026

Typosquats

14 packages harvesting AWS keys and CI/CD secrets.

AUG 2026

keyv / cacheable

400+ packages poisoned, 2B installs. Two days ago.

None of these are AWS vulnerabilities…

However, they all ended the same way: an attacker used a valid access key against your AWS account.

CloudTrail is a firehose, not a log file

1,000s

Distinct API calls

More events than anyone can read by hand.

GB / day

Gzipped JSON in S3

The evidence is already yours. Just not usable.

$ / GB

What ingest costs

Pricing decides which logs you get to keep.

Three challenges every responder hits

01

Cost decides your visibility

Trim log volume for the bill, and you drop exactly the events you'll want after a breach.

02

The tooling isn't where the logs are

Logs live in S3. Investigation happens somewhere else — or in jq and grep at 3 a.m.

03

You need AWS knowledge, SQL skills, and attacker knowledge all at once

The schema, hundreds of services, and knowing which of thousands of API calls matter.

Without these tools, here is what is left

PAY PER TB SCANNED

Amazon Athena

- Schema and partitions are your job
- Every exploratory query costs money
- A blank query box — zero detections

PAY PER GB INGESTED

A SIEM

- Ingest cost caps your retention
- Onboarding an account is a project
- Content is whatever the vendor shipped

PAY WITH YOUR NIGHT

jq / grep

- Breaks down at scale
- No correlation, no baseline, no severity
- Nothing is repeatable next time

All three require you to bring the knowledge.
None arrive with the community's detections loaded.

Open the logs — then what do you look for?

Which calls matter?

GetCallerIdentity is typically normal. From a new key, at 3 a.m., from a new ASN, it is not.

What is the chain?

Login without MFA → role enumeration → CloudTrail tampering. The sequence is the incident.

What is normal here?

Baselines are per-account. Nobody hands you one, and building it takes weeks.

The answers already exist. The community has been writing them down for years.

The logs are already yours.
What's missing is
the community's knowledge
and a way to run it on your laptop.

That is the entire design brief for Suzaku and Senrigan.

The Approach

We did not start from scratch.

We solved this problem once – for Windows event logs.



Hayabusa already proved the model

HAYABUSA — SINCE 2020

Windows event logs

- One Rust binary, no agent, no server
- Native Sigma rules from the community
- Runs offline on the analyst's machine
- Output is a timeline, not an alert queue

SUZAKU × SENRIGAN — NOW

AWS CloudTrail logs

- Same single-binary, offline philosophy
- Native Sigma, including correlation rules
- Docker Compose when you want a UI
- Azure and GCP support planned next

Three rules we refuse to break

速

Fast by default

Rust, multi-threaded, columnar. Scan .json.gz log files downloaded from your S3 bucket.

無

No infrastructure

One binary, or one command: **docker compose up**. Nothing leaves your machine.

共

Community knowledge

Every detection, query and chart comes from public community work.

The community already did the hard part

DETECTION RULES

SigmaHQ / Sigma

3,000+ peer-reviewed rules

TAXONOMY

MITRE ATT&CK Cloud

The shared vocabulary for cloud TTPs

TECHNIQUES

Threat Technique Catalog

What the AWS CIRT saw in real incidents

INCIDENT INTEL

TrailDiscover

CloudTrail events mapped to breaches

QUERIES

CloudTrail Lake samples

Battle-tested SQL, from aws-samples

PLAYBOOKS

aws-incident-response

IR procedures and queries from the field

VISUALIZATION

SIEM on Amazon OpenSearch

A reference for CloudTrail analytics

TEST DATA

flaws.cloud · Stratus · Pacu

Attack datasets we test every rule against

Rules, taxonomies, queries, dashboards, datasets — our job was to make it work together, offline.

Community knowledge, compiled into two tools

THE COMMUNITY
PUBLISHES

- SigmaHQ upstream rules
- ATT&CK Cloud techniques
- AWS CIRT catalog
- Query samples & playbooks

WE PACKAGE IT
INTO FILES

```
suzaku-rules
abused_aws_api_calls.csv
builtin_hunts.yaml
dashboard charts
```

YOU RUN IT
ON YOUR LAPTOP

```
./suzaku update-rules
./suzaku aws-ct-timeline
make ingest && make up
```

Pick a hunt. Get answers.

…and it flows back. False positives go upstream to SigmaHQ. Everyone's detections improve.

The usual way vs. the laptop way

THE USUAL WAY		WITH SUZAKU + SENRIGAN
Budget, procurement, a pipeline	SETUP	**One binary, or docker compose up**
Trimmed to fit the ingest bill	RETENTION	**Keep every event — storage is cheap**
Learn a vendor query language	QUERIES	**Sigma YAML and pre-built hunts**
Leaves your control	DATA	**100% offline, on your own machine**
Detections you rent	CONTENT	**Detections you own**

The line that matters is the last one: detection content you own, not rent.

Suzaku 朱雀

The vermilion bird that flies above the clouds.
Sigma rules straight onto CloudTrail.



One binary. Logs in, DFIR timeline out.

01

Standalone Rust CLI

No install, no runtime, no agent. Windows, Linux and macOS.

02

Reads .json, .json.gz, .parquet

Point it at an S3 export. Multi-threaded, nothing to unpack first.

03

Native Sigma detection

All field modifiers except expand, plus correlation rules.

04

Structured output

CSV, JSON, JSONL, DuckDB. Add -R to keep the raw JSON.

```
$ ./suzaku aws-ct-timeline \
    -d ./cloudtrail-logs \
    -o timeline.csv

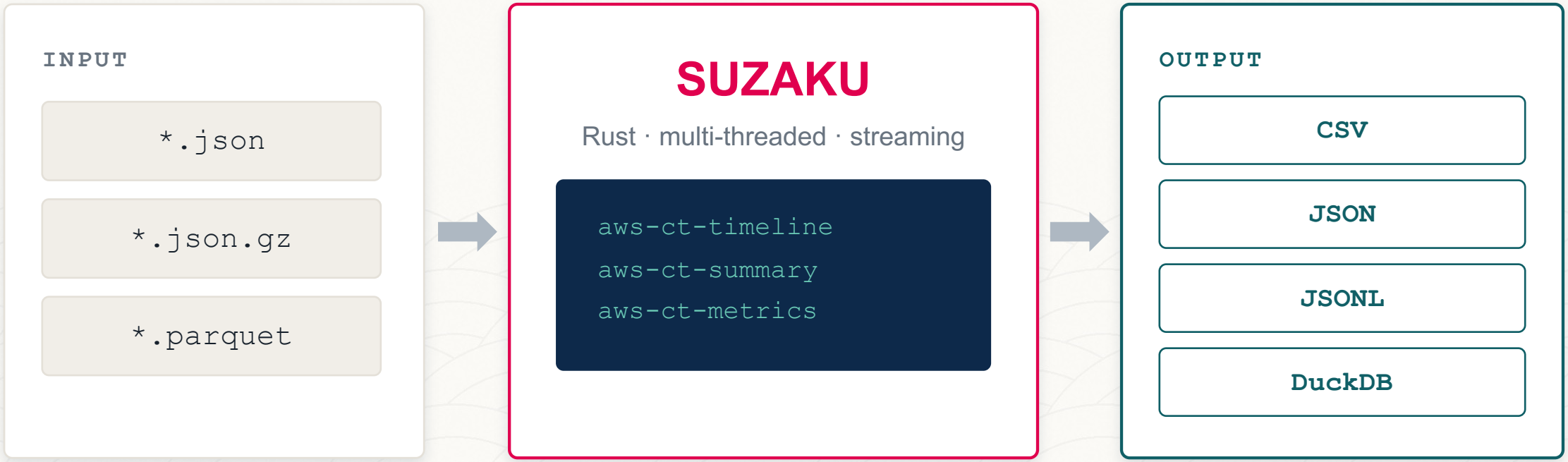
Rules loaded. Scanning...
```

AGPLv3

Rules under Detection Rule License 1.1

First released at Black Hat USA Arsenal 2025

Structured in, structured out



Open it anywhere: Excel · Timeline Explorer · pandas · Elastic Stack — or hand DuckDB to Senrigan.

Three commands cover the investigation

DFIR TIMELINE

aws-ct-timeline

Every Sigma rule applied, producing one chronological timeline of only what matters.

SIGNATURE-LESS

aws-ct-summary

Per-user profile: abused APIs, regions, IPs, user agents, access keys.

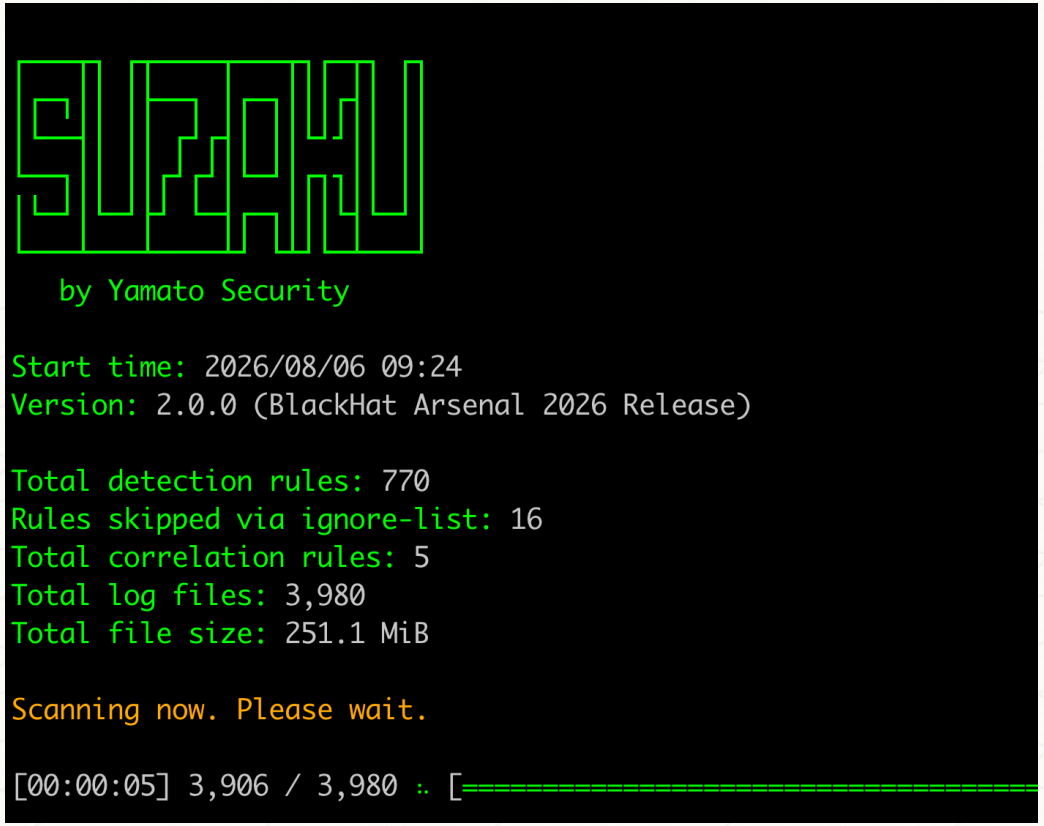
TRIAGE

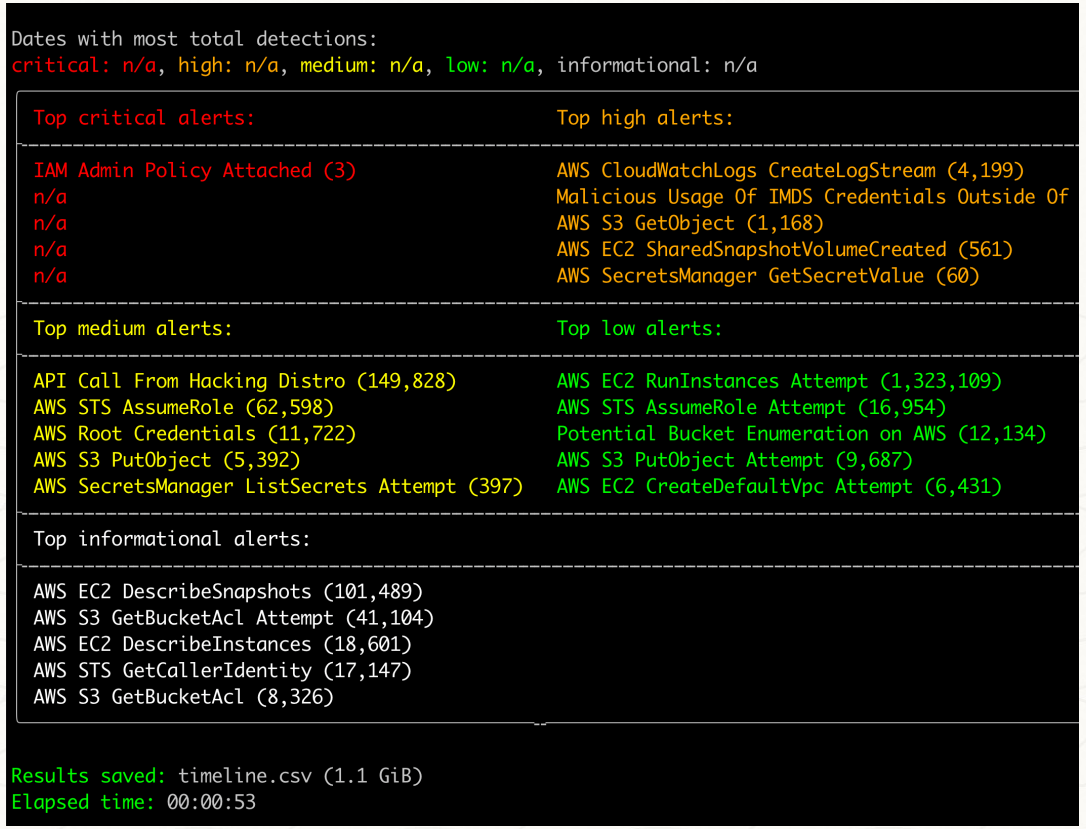
aws-ct-metrics

Frequency counts on any field. Baseline first, hunt second.

update-rules pulls the latest community rules — one command, no package manager.

The DFIR timeline





Detection frequency timeline, results summary, and one CSV you can hand to an analyst.

In the cloud, the sequence is the attack

event_count

Too many of the same event — password spraying, brute force.

value_count

Same event, many distinct values — one IP, many users.

temporal

Several rules firing in a timespan, any order — a recon burst.

temporal_ordered

The same, but in order — the attack chain.

```
correlation:
    type: temporal_ordered
    rules:
        - Console Login Without MFA
        - Role Enumeration
        - CloudTrail Settings Modified
    timespan: 1d
level: high
```

Three boring events. One high-severity finding.

suzaku-rules: community rules, kept current

Upstream Sigma, automatically

New SigmaHQ cloud rules land in suzaku-rules as they are merged.

Our own AWS rules

Where upstream coverage is thin, we write and publish — same license, for everyone.

abused_aws_api_calls.csv

Commonly abused API calls, with an explanation of why attackers use each one.

Your findings go back

Report a false positive and the fix reaches every Sigma user.

```
$ ./suzaku update-rules
```

Everything the community shipped since you last ran it.

aws-ct-summary: find what no rule caught

One row per user ARN — everything you need to spot a compromised identity.

AbusedAPIs-Success	AbusedAPIs-Failed	OtherAPIs-Success
AWS-Regions	SrcIPs	UserAgents
UserAccessKeyIDs	UserTypes	First / Last Seen

Real example: `aws-cli/1.16.301 Python/3.7.6 Linux/5.4.0-kali3-amd64`
The AWS CLI puts the OS in the user agent. No rule necessary — the summary just shows it.

Profiling one identity

UserARN	NumOfEvents	FirstTimestamp	LastTimestamp	AbusedAPIs-Success
arn:aws:iam::811596193553:user/backup	680,449	2017-02-12 21:15:12	2020-10-07 21:03:30	Unique APIs: 25 | Total API
arn:aws:iam::811596193553:user/Level6	651,646	2017-02-26 23:13:33	2020-10-07 14:52:39	Unique APIs: 27 | Total API
arn:aws:iam::811596193553:user/Security…	4,522	2017-05-26 22:25:09	2017-05-27 15:15:15	Unique APIs: 1 | Total APIs
arn:aws:iam::811596193553:root	2,686	2017-02-12 19:57:06	2020-10-07 15:52:02	Unique APIs: 10 | Total API
arn:aws:iam::123837392027:user/bert-jan	2,105	2023-07-10 11:54:33	2023-07-10 12:29:09	Unique APIs: 49 | Total API
arn:aws:iam::342082656213:user/Falsimen…	1,173	2021-07-29 18:03:04	2021-07-30 16:33:11	Unique APIs: 3 | Total APIs
arn:aws:sts::811596193553:assumed-role/…	304	2017-05-29 16:35:39	2017-05-30 00:35:45	Unique APIs: 5 | Total APIs
arn:aws:sts::811596193553:assumed-role/…	108	2019-08-21 08:05:34	2019-09-30 09:24:39	Unique APIs: 1 | Total APIs
arn:aws:iam::123837392027:user/benjamin	78	2023-07-10 11:42:18	2023-07-10 12:27:46	Unique APIs: 5 | Total APIs
arn:aws:iam::811596193553:user/piper	57	2017-05-16 03:38:22	2018-10-03 14:51:55	Unique APIs: 6 | Total APIs
arn:aws:iam::811596193553:user/Level5	39	2017-02-19 23:38:17	2017-02-23 04:09:21	Unique APIs: 2 | Total APIs
arn:aws:iam::342082656213:user/jmerckle	37	2021-07-29 13:02:53	2021-07-29 14:01:48	Unique APIs: 8 | Total APIs
arn:aws:sts::811596193553:assumed-role/…	22	2017-05-26 18:27:06	2017-05-26 18:30:40	Unique APIs: 1 | Total APIs
arn:aws:sts::811596193553:assumed-role/…	18	2017-03-11 03:00:35	2019-09-18 04:40:03	Unique APIs: 1 | Total APIs
arn:aws:iam::342082656213:root	5	2021-07-29 00:07:51	2021-07-30 10:37:34	Unique APIs: 1 | Total APIs
arn:aws:sts::123837392027:assumed-role/…	4	2023-07-10 12:15:59	2023-07-10 12:32:01	Unique APIs: 1 | Total APIs
arn:aws:iam::192374575148:user/TrailDis…	4	2024-08-18 12:59:16	2024-08-18 13:05:25	Unique APIs: 0
arn:aws:sts::811596193553:assumed-role/…	3	2017-05-29 16:35:27	2017-05-29 23:04:28	Unique APIs: 1 | Total APIs

No signatures involved. Everything this identity touched, in one row.

aws-ct-metrics: count values, on any field

```
$ ./suzaku aws-ct-metrics -d ./logs \
      -F eventName,userAgent -o metrics.csv
```

Any field — not just eventName

- `eventName (default)`
- `sourceIPAddress`
- `userAgent`
- `awsRegion`
- `errorCode`
- `userIdentity.type`

Frequency first

See what this account normally does before deciding what looks wrong.

The long tail is important

Rare user agents are more important to check than the top ones.

Drives our roadmap

We run it on sample data to pick which API call gets the next rule.

Your baseline, in one command

Field	Value	Count	Percent	FirstSeen	LastSeen
sourceIPAddress	5.205.62.253	1321919	67.01%	2019-08-21 07:41:17	2019-08-24 00:49:37
sourceIPAddress	ec2.amazonaws.com	44164	2.24%	2017-02-23 05:08:23	2023-07-10 12:03:26
sourceIPAddress	255.253.125.115	30197	1.53%	2017-02-12 19:57:06	2017-07-17 16:20:57
sourceIPAddress	193.29.252.218	23498	1.19%	2019-08-18 19:23:45	2020-10-07 21:03:30
sourceIPAddress	240.252.161.77	22394	1.14%	2019-06-17 17:50:41	2019-07-31 21:35:47
sourceIPAddress	0.52.31.206	18774	0.95%	2019-06-13 21:07:30	2019-08-02 17:45:34
sourceIPAddress	cloudtrail.amazonaws.com	17447	0.88%	2019-02-11 16:43:33	2023-07-10 12:08:09
sourceIPAddress	155.63.17.217	15925	0.81%	2018-04-16 06:59:18	2018-04-18 04:58:49
sourceIPAddress	24.251.252.2	10208	0.52%	2019-09-30 06:36:08	2019-09-30 07:41:38
sourceIPAddress	251.105.254.1	10106	0.51%	2020-06-10 04:58:48	2020-06-11 22:55:53
sourceIPAddress	delivery.logs.amazonaws.com	10011	0.51%	2021-07-29 23:58:37	2021-08-02 09:49:47
sourceIPAddress	255.171.1.252	9780	0.50%	2020-04-10 14:58:21	2020-09-22 14:18:26
sourceIPAddress	60.219.252.71	9216	0.47%	2020-09-11 16:29:39	2020-09-29 17:43:14
sourceIPAddress	253.0.255.253	9067	0.46%	2018-10-28 14:05:43	2018-10-29 04:59:14
sourceIPAddress	45.250.7.228	9060	0.46%	2019-06-21 06:38:18	2019-06-25 05:31:58

Read it from the top to learn the account. Read it from the bottom to find the attacker.

Senrigan 千里眼

The clairvoyant “thousand-mile seeing eye”.
Ask questions of your logs without writing SQL.



docker compose up, and you are hunting

01 **Four containers, one DuckDB file**

A Rust ingester writes; the UI, dashboard and graph read.

02 **Runs on a laptop**

Docker, make, 8 GB+ RAM, an SSD. No cluster, no license.

03 **Three URLs and you're in**

8501 to hunt, 8088 for dashboards, 8502 for the graph.

04 **Optional extras**

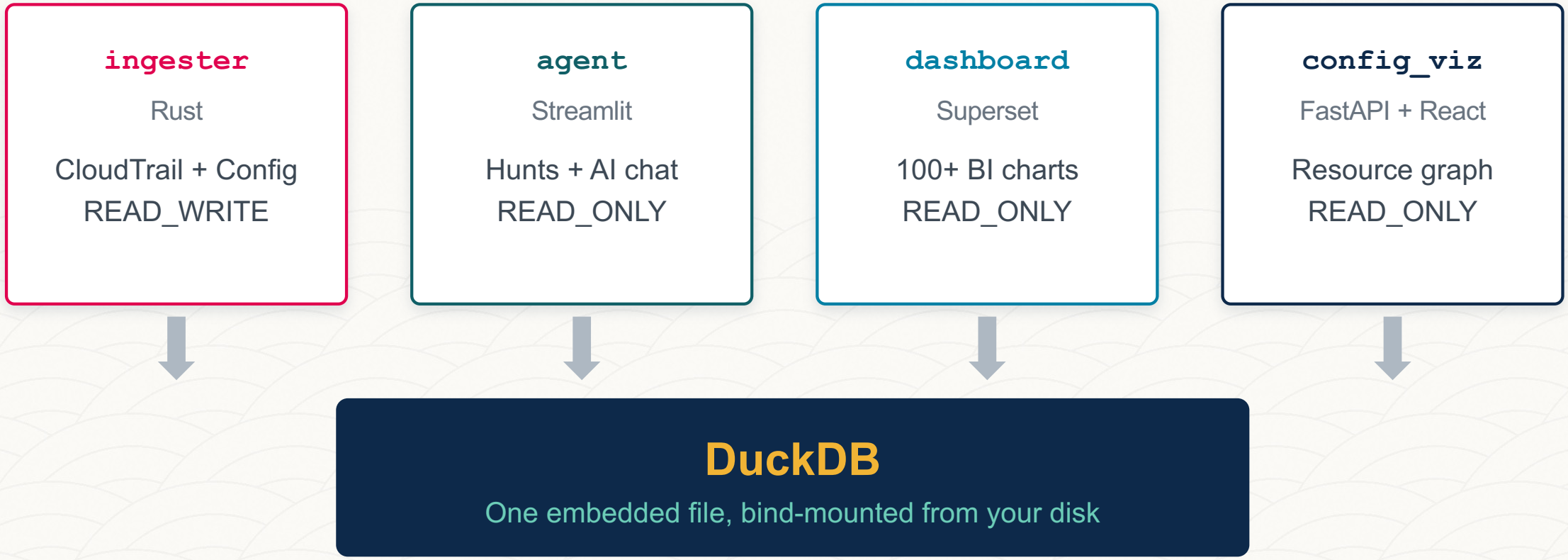
GeoLite2 for GeoIP, an OpenAI key for AI.

```
$ git clone .../senrigan
$ cp -r ./logs docker/logs/
$ make ingest
$ make up

→ http://localhost:8501
→ http://localhost:8088
```

Nothing leaves your machine.

One writer, three readers, zero servers



Columnar analytics on a laptop — this is what makes “no SIEM” possible.

100+ hunts. No SQL, no need to learn the schema.

26 🔑

Identity & Access

Root, MFA, PassRole, privesc

18 🪣

Data & Storage

S3 bulk download, snapshots

15 🌐

Network & Infra

SG open, flow log deletion

13 ⚡

Compute

Lambda, SSM, ECS backdoor

10 🛡️

Defense Tampering

GuardDuty, Config, CloudTrail

10 🌍

GeoIP Analysis

Impossible travel, countries

5 🕵️

Threat Patterns

Off-hours, recon bursts

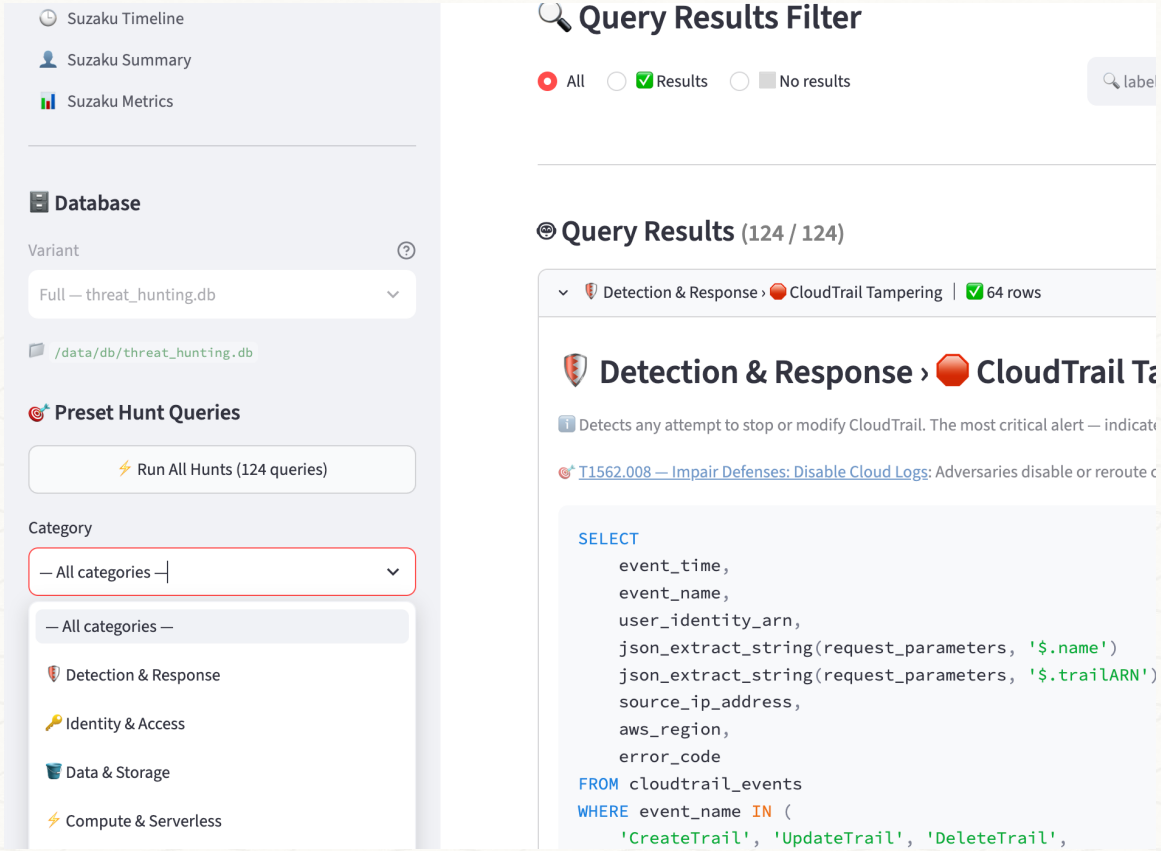
4 📊

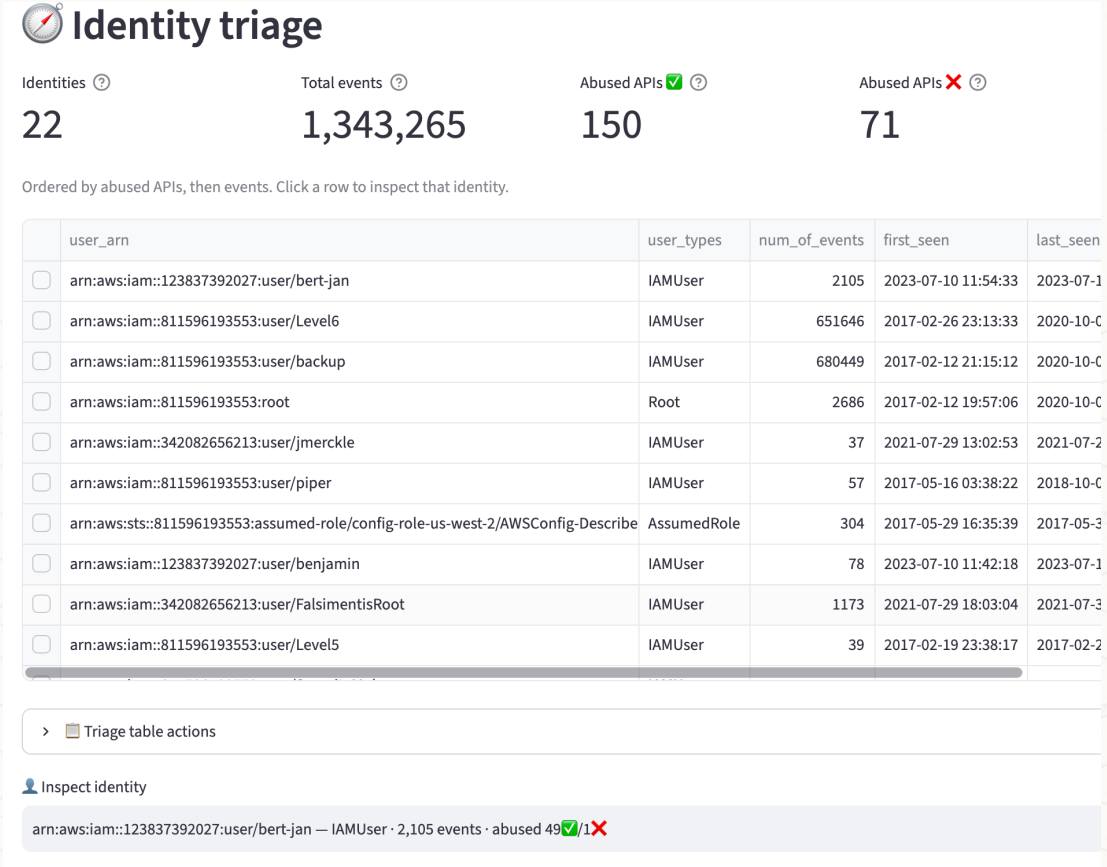
Baseline & IaC

Error spikes, CI/CD abuse

Pick one from a dropdown. The SQL is there if you want to read or edit it.

Pick a hunt, get an answer





Dropdown, results, and the SQL query underneath if you want it.

100+ pre-built charts across 8 tabs

- 🎯 **Threat Detection**
- 🔑 **Identity & Access**
- 🪣 **S3 & RDS**
- 📊 **API Activity**
- 🚨 **High-Risk APIs**
- 🕒 **Temporal Analysis**
- 🖥️ **Computing**
- 🌍 **GeoIP Intelligence**

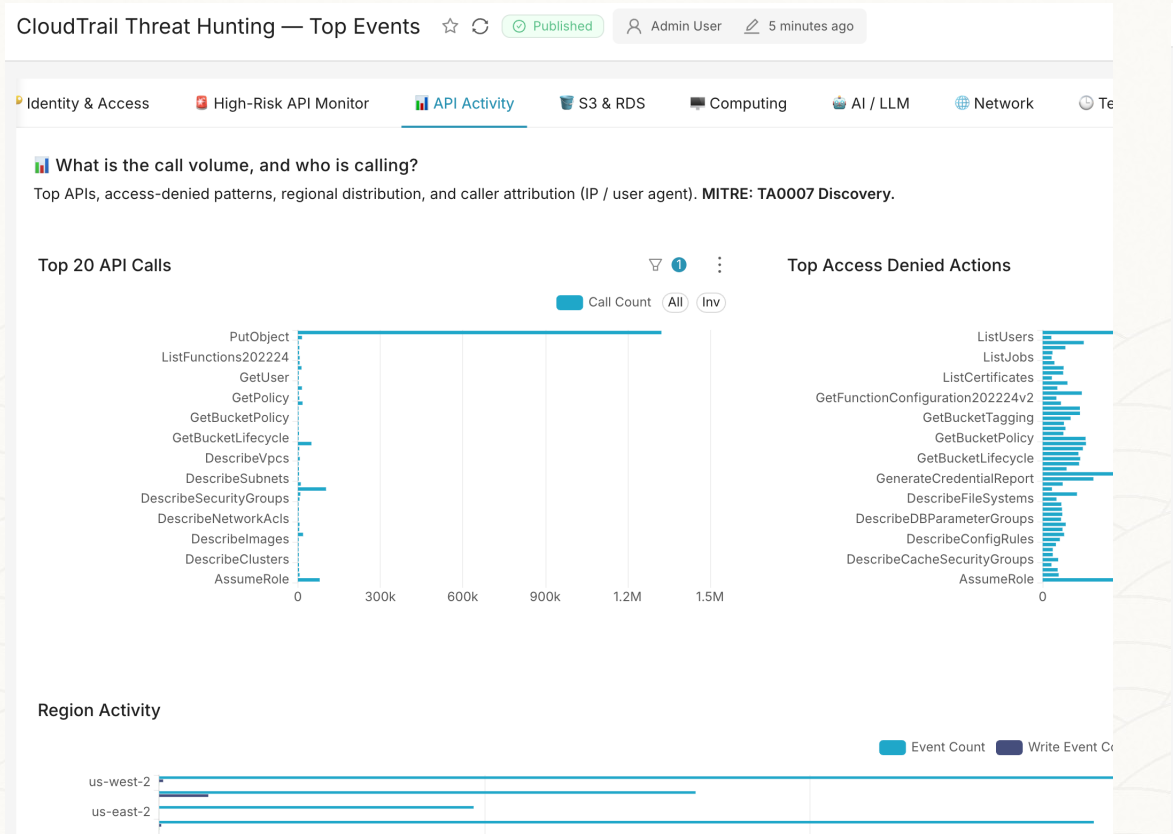
What you get for free

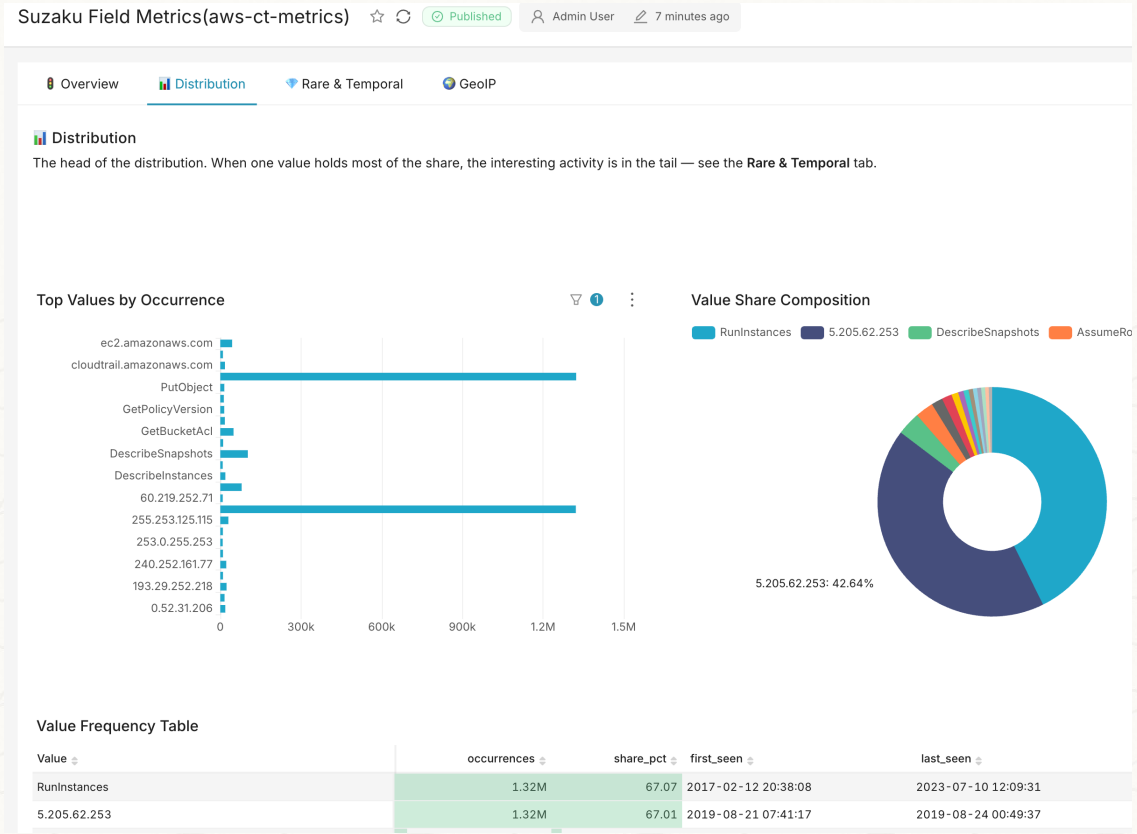
First / last seen per identity, IP, API and user agent

Dormant accounts that suddenly woke up after 72 hours

Login heatmap by hour and day, and a world map of origins

100+ charts you did not have to build





Every chart is a Superset asset in the repo — fork it, edit it, add your own.

For questions without a pre-built hunt

AI chat over your logs

Ask in plain language; the agent writes and runs the SQL, read-only.

HTML hunting report

One portable file for management, a ticket, or the case archive.

AWS Config resource graph

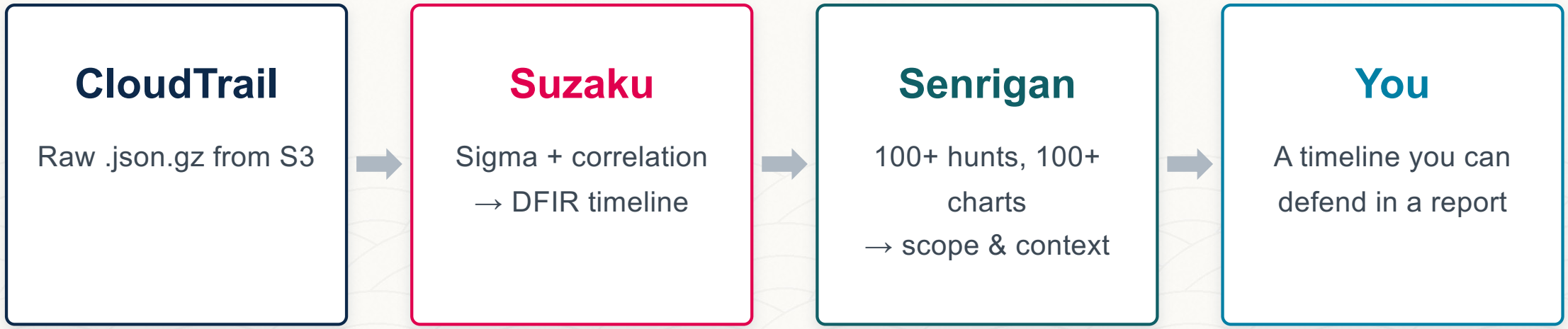
See how resources connect:
context CloudTrail alone can't give you.

GeoIP enrichment

Country, city and ASN on every IP.
This is what powers impossible travel.

All local. The only thing that leaves your machine is an AI prompt — if you opt in.

Suzaku finds it. Senrigan explains it.



One tells you what fired. The other tells you how far it went.

Live Demo

Real attack data from flaws.cloud.
Raw logs to a finished investigation.



Four things you are about to see

01 **Detect**

```
./suzaku aws-ct-timeline -d ../suzaku-sample-data
```

02 **Pivot**

```
./suzaku aws-ct-summary -d ../suzaku-sample-data
```

03 **Ingest**

```
make ingest && make up
```

04 **Hunt**

```
open http://localhost:8501
```

Data: flaws.cloud — a public CloudTrail dataset with real attack activity inside it.

TAKE IT HOME

Free, open source, and ready to run today

01 You already have the logs. Start with the binary.

02 The detections are the community's — and yours.

03 No SIEM, no cloud, no license. Just your laptop.

QUESTIONS?

Thank you

github.com/Yamato-Security/senrigan

github.com/Yamato-Security/suzaku

感謝 — with many thanks to SigmaHQ, MITRE, the AWS community and every project we built on.